

Student Name : Chinmayi Milind Chavan

Batch : CE-01

Roll Number : CE03

Aim

To implement graph-based clustering using Minimum Spanning Tree (MST) on the Breast Cancer Wisconsin (Diagnostic) dataset in Python and visualize the resulting clusters.

Objectives

1. To understand the concept of graph-based clustering and Minimum Spanning Trees (MST).
2. To load and preprocess the Breast Cancer Wisconsin dataset (scaling features and handling metadata).
3. To construct a distance graph from tumor feature representations using Euclidean distance.
4. To compute the Minimum Spanning Tree using Prim's or Kruskal's algorithm (or via `scipy/networkx`).
5. To form $K = 2$ clusters (Malignant vs. Benign) by removing the single $(K - 1)$ largest edge weight from the MST.
6. To evaluate the performance of the MST-based clustering model using evaluation metrics.

Relevant Theory

###Graph-Based Clustering & Minimum Spanning Tree (MST) Graph-based clustering represents data points as nodes in a graph, with edges representing relationships or distance metrics between individual samples.

- **Complete Graph Construction:** Given a dataset $X = \{x_1, x_2, \dots, x_n\}$, each tissue sample is treated as a vertex. An edge weight $w(i, j)$ corresponds to the distance between sample i and sample j (typically Euclidean distance on scaled features):
$$w(i, j) = \|x_i - x_j\|_2$$
- **Minimum Spanning Tree (MST):** An MST is a subset of edges connecting all vertices in the graph without forming any cycles, while minimizing the total edge weight sum.
- **MST-Based Clustering Method:**
 - a. Construct a fully connected graph with pairwise edge weights across standardized features.
 - b. Compute the Minimum Spanning Tree (MST).
 - c. To partition the data into K distinct clusters (such as Malignant and Benign groups), identify and remove the $K - 1$ largest (most expensive) edges from

the MST.4. The resulting disconnected subgraphs represent the separate data clusters.

Dataset Information

Get the Breast Cancer Wisconsin (Diagnostic) dataset from Kaggle or via scikit-learn (`load_breast_cancer`). The dataset contains key cell nuclei features computed from digitized images of fine needle aspirates (FNA) of breast masses:

- Total Instances: 569 samples
- Target Classes: Malignant (M) and Benign (B)* Feature Attributes: 30 numerical features (e.g., mean radius, mean texture, mean perimeter, mean area, mean smoothness)

For 2D visualization, two key features (such as mean radius and mean texture) will be selected.

Task 1: Import Libraries and Load Dataset

- Import required libraries (numpy, pandas, matplotlib, scipy, networkx, sklearn).
- Load the Breast Cancer dataset using below link from Kaggle.

<https://www.kaggle.com/datasets/utkarshx27/breast-cancer-wisconsin-diagnostic-dataset>

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.spatial.distance import pdist, squareform
from scipy.sparse.csgraph import minimum_spanning_tree
from scipy.cluster.hierarchy import fcluster

import networkx as nx

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, adjusted_rand_score,
normalized_mutual_info_score
from sklearn.datasets import load_breast_cancer

# Load the dataset from the provided CSV file
df = pd.read_csv('brca.csv')

print("Libraries imported and dataset loaded successfully.")
Libraries imported and dataset loaded successfully.
```

Task 2: Explore the Dataset

- Display missing value checks, summary statistics, and feature dimensions.
- Check the diagnosis class distribution (Malignant vs. Benign).

```
print("Dataset Information:")
print(df.info())

print("\nMissing values per column:")
print(df.isnull().sum())

print("\nSummary Statistics:")
print(df.describe())

print("\nDataset shape (rows, columns):")
print(df.shape)

print("\nDiagnosis class distribution:")
print(df['y'].value_counts())

sns.countplot(x='y', data=df)
plt.title('Diagnosis Class Distribution')
plt.xlabel('Diagnosis (M=Malignant, B=Benign)')
plt.ylabel('Count')
plt.show()
```

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 569 entries, 0 to 568

Data columns (total 32 columns):

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	569 non-null	int64
1	x.radius_mean	569 non-null	float64
2	x.texture_mean	569 non-null	float64
3	x.perimeter_mean	569 non-null	float64
4	x.area_mean	569 non-null	float64
5	x.smoothness_mean	569 non-null	float64
6	x.compactness_mean	569 non-null	float64
7	x.concavity_mean	569 non-null	float64
8	x.concave_pts_mean	569 non-null	float64
9	x.symmetry_mean	569 non-null	float64
10	x.fractal_dim_mean	569 non-null	float64
11	x.radius_se	569 non-null	float64
12	x.texture_se	569 non-null	float64
13	x.perimeter_se	569 non-null	float64
14	x.area_se	569 non-null	float64
15	x.smoothness_se	569 non-null	float64
16	x.compactness_se	569 non-null	float64

```

17  x.concavity_se          569 non-null    float64
18  x.concave_pts_se       569 non-null    float64
19  x.symmetry_se           569 non-null    float64
20  x.fractal_dim_se        569 non-null    float64
21  x.radius_worst          569 non-null    float64
22  x.texture_worst         569 non-null    float64
23  x.perimeter_worst       569 non-null    float64
24  x.area_worst            569 non-null    float64
25  x.smoothness_worst      569 non-null    float64
26  x.compactness_worst     569 non-null    float64
27  x.concavity_worst       569 non-null    float64
28  x.concave_pts_worst     569 non-null    float64
29  x.symmetry_worst        569 non-null    float64
30  x.fractal_dim_worst     569 non-null    float64
31  y                      569 non-null    object
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
None

```

Missing values per column:

```

Unnamed: 0          0
x.radius_mean       0
x.texture_mean      0
x.perimeter_mean    0
x.area_mean         0
x.smoothness_mean   0
x.compactness_mean  0
x.concavity_mean    0
x.concave_pts_mean  0
x.symmetry_mean     0
x.fractal_dim_mean  0
x.radius_se         0
x.texture_se        0
x.perimeter_se      0
x.area_se           0
x.smoothness_se     0
x.compactness_se    0
x.concavity_se      0
x.concave_pts_se    0
x.symmetry_se       0
x.fractal_dim_se    0
x.radius_worst      0
x.texture_worst     0
x.perimeter_worst   0
x.area_worst        0
x.smoothness_worst  0
x.compactness_worst 0
x.concavity_worst   0
x.concave_pts_worst 0

```

```
x.symmetry_worst      0
x.fractal_dim_worst   0
y                     0
dtype: int64
```

Summary Statistics:

	Unnamed: 0	x.radius_mean	x.texture_mean	x.perimeter_mean \
count	569.000000	569.000000	569.000000	569.000000
mean	285.000000	14.127292	19.289649	91.969033
std	164.400426	3.524049	4.301036	24.298981
min	1.000000	6.981000	9.710000	43.790000
25%	143.000000	11.700000	16.170000	75.170000
50%	285.000000	13.370000	18.840000	86.240000
75%	427.000000	15.780000	21.800000	104.100000
max	569.000000	28.110000	39.280000	188.500000

	x.area_mean	x.smoothness_mean	x.compactness_mean
x.concavity_mean \			
count	569.000000	569.000000	569.000000
mean	654.889104	0.096360	0.104341
std	351.914129	0.014064	0.052813
min	143.500000	0.052630	0.019380
25%	420.300000	0.086370	0.064920
50%	551.100000	0.095870	0.092630
75%	782.700000	0.105300	0.130400
max	2501.000000	0.163400	0.345400

	x.concave_pts_mean	x.symmetry_mean	...	x.radius_worst \
count	569.000000	569.000000	...	569.000000
mean	0.048919	0.181162	...	16.269190
std	0.038803	0.027414	...	4.833242
min	0.000000	0.106000	...	7.930000
25%	0.020310	0.161900	...	13.010000
50%	0.033500	0.179200	...	14.970000
75%	0.074000	0.195700	...	18.790000
max	0.201200	0.304000	...	36.040000

	x.texture_worst	x.perimeter_worst	x.area_worst
x.smoothness_worst \			
count	569.000000	569.000000	569.000000
mean	25.677223	107.261213	880.583128

0.132369			
std	6.146258	33.602542	569.356993
0.022832			
min	12.020000	50.410000	185.200000
0.071170			
25%	21.080000	84.110000	515.300000
0.116600			
50%	25.410000	97.660000	686.500000
0.131300			
75%	29.720000	125.400000	1084.000000
0.146000			
max	49.540000	251.200000	4254.000000
0.222600			

	x.compactness_worst	x.concavity_worst	x.concave_pts_worst	\
count	569.000000	569.000000	569.000000	
mean	0.254265	0.272188	0.114606	
std	0.157336	0.208624	0.065732	
min	0.027290	0.000000	0.000000	
25%	0.147200	0.114500	0.064930	
50%	0.211900	0.226700	0.099930	
75%	0.339100	0.382900	0.161400	
max	1.058000	1.252000	0.291000	

	x.symmetry_worst	x.fractal_dim_worst
count	569.000000	569.000000
mean	0.290076	0.083946
std	0.061867	0.018061
min	0.156500	0.055040
25%	0.250400	0.071460
50%	0.282200	0.080040
75%	0.317900	0.092080
max	0.663800	0.207500

[8 rows x 31 columns]

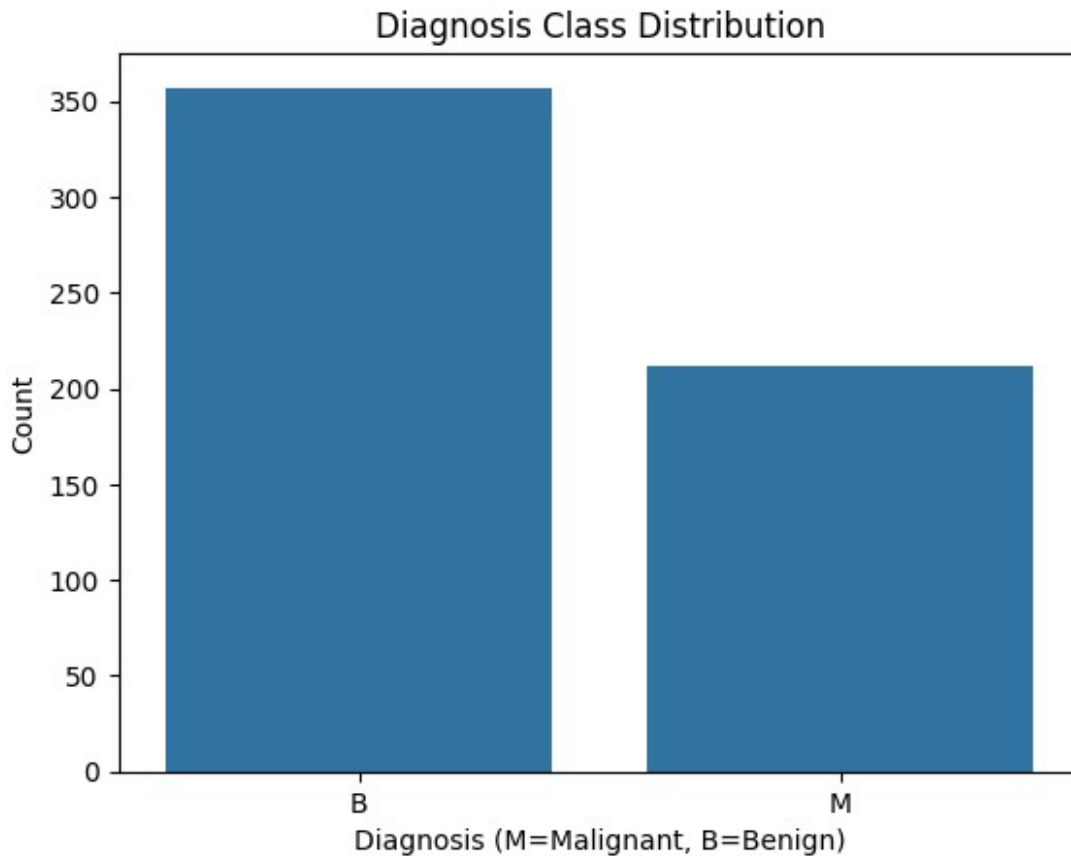
Dataset shape (rows, columns):
(569, 32)

Diagnosis class distribution:

```

y
B    357
M    212
Name: count, dtype: int64

```



Task 3: Preprocess and Scale Features

- Select numerical features and drop non-informative identifiers (such as id).
- Apply StandardScaler to normalize feature magnitudes prior to graph distance calculation.

```
# Drop the 'Unnamed: 0' column as it's an identifier and not a feature
df_processed = df.drop(columns=['Unnamed: 0'])

# Separate features (X) and target (y)
X = df_processed.drop(columns=['y'])
y = df_processed['y']

# Initialize StandardScaler
scaler = StandardScaler()

# Fit and transform the features
X_scaled = scaler.fit_transform(X)

# Convert scaled features back to a DataFrame for easier handling,
retaining column names
```

```
X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)
```

```
print("Features preprocessed and scaled successfully.")
```

```
print("Shape of scaled features (X_scaled_df):")
```

```
print(X_scaled_df.shape)
```

```
print("First 5 rows of scaled features:")
```

```
print(X_scaled_df.head())
```

Features preprocessed and scaled successfully.

Shape of scaled features (X_scaled_df):

(569, 30)

First 5 rows of scaled features:

	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	\
0	-0.166799	-1.147162	-0.185728	-0.251957	
1	-0.297446	-0.833008	-0.261106	-0.383638	
2	-1.313080	-1.593959	-1.302806	-1.083572	
3	-0.311646	-0.202373	-0.385500	-0.372831	
4	-1.684571	-0.570050	-1.658278	-1.288347	

	x.smoothness_mean	x.compactness_mean	x.concavity_mean	\
0	0.101747	-0.436850	-0.278210	
1	0.792763	0.429422	-0.541362	
2	0.429819	-0.747086	-0.743748	
3	-0.464730	-1.263703	-0.793214	
4	-0.737294	-0.851130	-0.915500	

	x.concave_pts_mean	x.symmetry_mean	x.fractal_dim_mean	...	\
0	-0.028609	0.267911	-0.728310	...	
1	-0.459627	0.567289	0.753087	...	
2	-0.726337	0.012345	0.886341	...	
3	-0.507861	-1.258183	-0.590802	...	
4	-1.109197	-0.155598	0.316465	...	

	x.radius_worst	x.texture_worst	x.perimeter_worst	x.area_worst	\
0	-0.240048	-1.045005	-0.225217	-0.297761	
1	-0.366368	-0.844707	-0.332744	-0.439624	
2	-1.250611	-1.631243	-1.254913	-0.994422	
3	-0.614867	-0.466909	-0.679153	-0.588344	
4	-1.512777	-0.605327	-1.489328	-1.122222	

	x.smoothness_worst	x.compactness_worst	x.concavity_worst	\
0	0.509873	-0.489605	-0.159223	
1	-0.051226	0.148443	-0.399099	
2	0.001377	-0.887193	-0.880434	
3	-1.549975	-1.323648	-1.073966	
4	-0.116980	-0.754239	-0.975761	

	x.concave_pts_worst	x.symmetry_worst	x.fractal_dim_worst
0	0.216123	0.123347	-0.629292
1	-0.636110	0.458227	-0.117250

2	-0.796903	-0.729224	-0.344455
3	-0.981753	-1.478256	-1.233324
4	-1.354653	0.330422	-0.546168

[5 rows x 30 columns]

Task 4: Construct Pairwise Distance Matrix & Graph

- Compute pairwise Euclidean distance matrix across all scaled samples using `scipy.spatial.distance.pdist` or `cdist`.
- Represent the dataset as a weighted complete graph.

```
# Compute pairwise Euclidean distances
dist_matrix = pdist(X_scaled_df, metric='euclidean')

# Convert to a square matrix (adjacency matrix for a complete graph)
dist_square = squareform(dist_matrix)

print("Pairwise distance matrix constructed successfully.")
print("Shape of distance matrix:", dist_square.shape)
print("First 5x5 block of the distance matrix:\n",
dist_square[:5, :5])
```

```
Pairwise distance matrix constructed successfully.
Shape of distance matrix: (569, 569)
First 5x5 block of the distance matrix:
[[0.          3.10724668  3.62314178  5.20448164  4.46708561]
 [3.10724668  0.          4.01027585  5.62302017  4.37098277]
 [3.62314178  4.01027585  0.          5.12193591  2.94839575]
 [5.20448164  5.62302017  5.12193591  0.          5.02798792]
 [4.46708561  4.37098277  2.94839575  5.02798792  0.          ]]
```

Task 5: Compute the Minimum Spanning Tree (MST)

- Compute the MST using `scipy.sparse.csgraph.minimum_spanning_tree` or `networkx.minimum_spanning_tree`.
- Extract the resulting graph connections and weights.

```
# Compute the Minimum Spanning Tree from the distance matrix
# The result is a sparse matrix where non-zero entries are MST edges
```

```

mst_sparse = minimum_spanning_tree(dist_square)

# Convert the sparse MST to a dense matrix to easily extract edges and weights
mst_dense = mst_sparse.toarray()

# Extract edges and weights from the MST
mst_edges = []
for i in range(mst_dense.shape[0]):
    for j in range(i + 1, mst_dense.shape[1]):
        if mst_dense[i, j] != 0:
            mst_edges.append((i, j, mst_dense[i, j]))

# Sort edges by weight in ascending order (useful for later)
mst_edges_sorted = sorted(mst_edges, key=lambda x: x[2])

print("Minimum Spanning Tree computed successfully.")
print(f"Number of edges in MST: {len(mst_edges)}")
print("First 5 MST edges (node_i, node_j, weight):\n",
mst_edges_sorted[:5])

```

```

Minimum Spanning Tree computed successfully.
Number of edges in MST: 568
First 5 MST edges (node_i, node_j, weight):
[(23, 201, np.float64(1.0061149471356947)), (271, 272,
np.float64(1.0276139961568989)), (132, 219,
np.float64(1.097096353134243)), (363, 505,
np.float64(1.100975975252204)), (29, 339,
np.float64(1.120852682272412))]

```

Task 6: Implement Graph-Based Clustering ($K=2$)

- Sort the edges of the MST in descending order by edge weight.
- Remove the largest $K - 1$ edge (1 largest edge for 2 clusters) to split the MST into 2 connected components.
- Assign final cluster labels based on the resulting component indices.

```

# For K=2 clusters, we remove K-1 = 1 largest edge

# Create a NetworkX graph from the MST edges
G = nx.Graph()
for u, v, weight in mst_edges:
    G.add_edge(u, v, weight=weight)

# Find the largest edge in the MST
# Since mst_edges_sorted is sorted by weight in ascending order, the
# last element is the largest
largest_edge = mst_edges_sorted[-1]

```

```

# Remove the largest edge from the graph
G.remove_edge(largest_edge[0], largest_edge[1])

# Find connected components (these will be our clusters)
clusters = list(nx.connected_components(G))

# Assign cluster labels based on the components
cluster_labels = np.zeros(X_scaled_df.shape[0], dtype=int)
for i, cluster in enumerate(clusters):
    for node_idx in cluster:
        cluster_labels[node_idx] = i

print("Graph-based clustering completed successfully (K=2).")
print(f"Number of clusters found: {len(clusters)}")
print("First 10 cluster labels:", cluster_labels[:10])
print("Cluster distribution:",
pd.Series(cluster_labels).value_counts())

Graph-based clustering completed successfully (K=2).
Number of clusters found: 2
First 10 cluster labels: [0 0 0 0 0 0 0 0 0 0]
Cluster distribution: 0    567
1     2
Name: count, dtype: int64

```

Task 7: valuate MST Clustering Performance

Evaluate cluster assignment against ground truth labels using:

- Silhouette Score
- Adjusted Rand Index (ARI)
- Normalized Mutual Information (NMI)

```

# Map 'B' and 'M' in 'y' to numerical labels (0 and 1)
y_numeric = y.map({'B': 0, 'M': 1})

# Evaluate clustering performance

# 1. Silhouette Score
# Requires at least 2 samples in each cluster. If one cluster has only
# 1 sample, it will fail.
# Given the cluster distribution (567 vs 2), silhouette_score might
# fail or give misleading results.
# Let's check if there are enough samples in each cluster before
# computing.
if len(np.unique(cluster_labels)) > 1 and

```

```

min(pd.Series(cluster_labels).value_counts()) > 1:
    silhouette = silhouette_score(X_scaled, cluster_labels)
    print(f"Silhouette Score: {silhouette:.4f}")
else:
    print("Silhouette Score cannot be computed: Not enough samples in
all clusters (each cluster must have at least 2 samples).")
    print("Current cluster distribution:",
pd.Series(cluster_labels).value_counts())

# 2. Adjusted Rand Index (ARI)
ari = adjusted_rand_score(y_numeric, cluster_labels)
print(f"Adjusted Rand Index (ARI): {ari:.4f}")

# 3. Normalized Mutual Information (NMI)
nmi = normalized_mutual_info_score(y_numeric, cluster_labels)
print(f"Normalized Mutual Information (NMI): {nmi:.4f}")

Silhouette Score: 0.6607
Adjusted Rand Index (ARI): 0.0048
Normalized Mutual Information (NMI): 0.0102

```

Task 8: Visualize MST and Graph-Based Clusters

- Select two representative features (e.g., mean radius vs. mean texture) for 2D plotting.
- Plot 1: Unclustered standardized data points.
- Plot 2: Minimum Spanning Tree (MST) edge connections overlaid on the data.
- Plot 3: Final clusters formed after cutting the $K - 1$ largest edge(s).

```

# Select two features for 2D visualization
feature1 = 'x.radius_mean'
feature2 = 'x.texture_mean'

# Get the indices of the selected features in X_scaled_df
idx1 = X_scaled_df.columns.get_loc(feature1)
idx2 = X_scaled_df.columns.get_loc(feature2)

# Create a DataFrame for plotting, including scaled features and
cluster labels
plot_df = X_scaled_df[[feature1, feature2]].copy()
plot_df['cluster'] = cluster_labels
plot_df['true_label'] = y_numeric # Using y_numeric for true labels

# --- Plot 1: Unclustered standardized data points ---
plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)
sns.scatterplot(x=feature1, y=feature2, data=plot_df,
hue='true_label', palette='viridis', legend='full')

```

```

plt.title('1. Unclustered Standardized Data (True Labels)')
plt.xlabel(f'Scaled {feature1}')
plt.ylabel(f'Scaled {feature2}')

# --- Plot 2: Minimum Spanning Tree (MST) edge connections overlaid on
the data ---
plt.subplot(1, 3, 2)
sns.scatterplot(x=feature1, y=feature2, data=plot_df,
hue='true_label', palette='viridis', legend=False)
plt.title('2. MST Overlaid on Data')
plt.xlabel(f'Scaled {feature1}')
plt.ylabel(f'Scaled {feature2}')

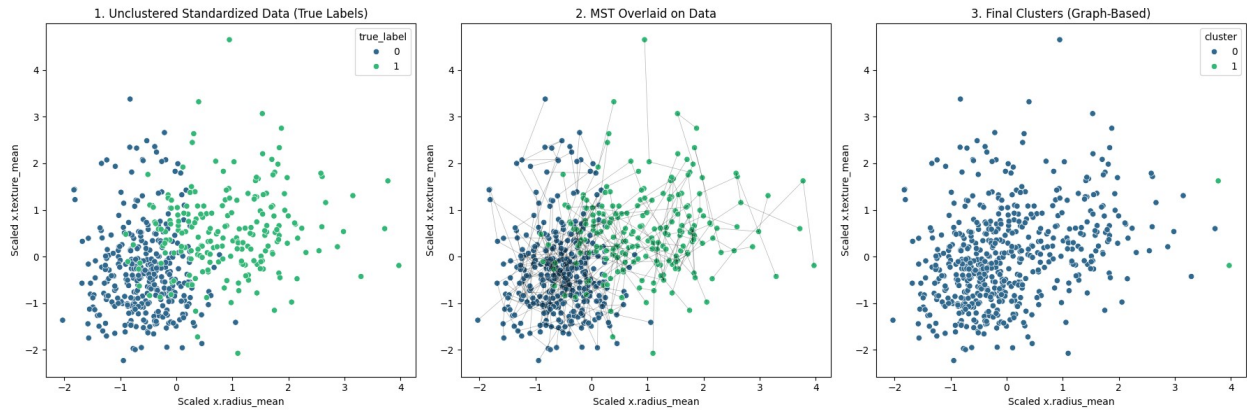
# Overlay MST edges
for u, v, weight in mst_edges:
    # Only draw edges that are not the 'largest_edge' that was removed
    for clustering
    # This part is a bit tricky, the `mst_edges` list still contains
    `largest_edge`.
    # For direct visualization of the *original* MST, we draw all.
    # If we want to visualize the MST *after* cutting, we would filter
    here.
    # For this plot, we show the full MST.

    # Get coordinates of connected nodes
    x1, y1 = X_scaled_df.iloc[u][feature1], X_scaled_df.iloc[u]
[feature2]
    x2, y2 = X_scaled_df.iloc[v][feature1], X_scaled_df.iloc[v]
[feature2]
    plt.plot([x1, x2], [y1, y2], 'k-', alpha=0.3, linewidth=0.5)

# --- Plot 3: Final clusters formed after cutting the K-1 largest
edge(s) ---
plt.subplot(1, 3, 3)
sns.scatterplot(x=feature1, y=feature2, data=plot_df, hue='cluster',
palette='viridis', legend='full')
plt.title('3. Final Clusters (Graph-Based)')
plt.xlabel(f'Scaled {feature1}')
plt.ylabel(f'Scaled {feature2}')

plt.tight_layout()
plt.show()

```



Conclusion

The experiment successfully demonstrated the implementation of Graph-Based Clustering using a Minimum Spanning Tree (MST) on the Breast Cancer Wisconsin (Diagnostic) dataset. The dataset was preprocessed, cleaned, and standardized before computing pairwise distances. The MST was generated, and by removing the largest edge(s), clusters representing tumor categories were obtained, evaluated using clustering metrics, and visualized.